

AEM: Overlay, Override & sling:resourceSuperType

A complete guide to AEM's component customization strategies

1. Overlay

Copying a node from **/libs** to **/apps** at the **same path**, then modifying it. AEM's Sling Resource Merger merges the two, with **/apps** taking precedence.

Example path:

```
/libs/wcm/core/content/siteadmin  
→ /apps/wcm/core/content/siteadmin
```

How it works:

Sling Resource Merger merges properties from both paths, so you only need to define what you want to change — everything else is inherited from **/libs**.

Use cases:

- Customizing OOTB components, dialogs, or console UI
- Changing Touch UI configurations
- Modifying clientlibs or HTL scripts

2. Override

Completely replacing a resource by placing a node at a **different path** that takes full precedence, or replacing a **Java/OSGi service** with a higher-ranked implementation.

OSGi Service Ranking Example:

```
@Component(service = ImageImpl.class, property = {  
    "service.ranking:Integer=100" // higher than default (0)  
})  
  
public class CustomImageImpl extends ImageImpl {  
    @Override  
    public String getSrc() {  
        return "https://cdn.example.com/" + super.getSrc();  
    }  
}
```

How OSGi ranking works:

Class	service.ranking	OSGi picks?
AEM's ImageImpl	0 (default)	X
Your CustomImageImpl	100	✓ YES

3. sling:resourceSuperType

A form of **override with inheritance** — your component lives at a **different path** and inherits everything from the parent, overriding only what you define.

Configuration:

```
/apps/myproject/components/myimage
sling:resourceSuperType = "core/wcm/components/image/v3/image"
```

File structure — only override what you need:

```
/apps/myproject/components/myimage/
■■■ .content.xml ← sets sling:resourceSuperType
■■■ myimage.html ← your custom HTL (overrides parent)

# Everything else (dialog, policies) inherited from core image component
```

Key insight: You only need to add the files/scripts you want to change. AEM automatically pulls everything else from the parent component defined in sling:resourceSuperType.

4. Comparison: Overlay vs Override vs resourceSuperType

Aspect	Overlay	Override	resourceSuperType
Mechanism	Sling Resource Merger	OSGi service ranking	Sling inheritance
Path	Same path under /apps	Different path / new class	Different path
Inheritance	Partial (merges with /libs)	None (full replace)	Full (inherits all)
Scope	All uses of that node	All uses of that service	Only your component
Use case	UI, dialogs, scripts	Backend Java logic	Component inheritance
Upgrade safety	High	Medium	High

■ **The Golden Rule:** Never modify **/libs** directly. Always overlay into **/apps**. This keeps your changes upgrade-safe, since AEM updates only affect **/libs**. Use sling:resourceSuperType for new components and OSGi override for backend Java logic.